

Sharif University of Technology
Department of Mechanical Engineering

Path Planning Using Dijkstra's Algorithm

Robotics Course – Homework 8

Haniyeh Zahra Budaqi – 99105956
Instructor: Dr. Behzadipour

First Term, 1402

Translator's note: The source PDF for this assignment (`report_fa.pdf`) was found, on inspection of its actual content, to be a verbatim copy of the report submitted for Homework 7 ("Path Planning Using Dijkstra's Algorithm"), not the Homework 8 Simulink modeling assignment. The title page of the original document itself reads "Homework 7 of the Robotics course." This translation faithfully reproduces the content of the source file exactly as found, in keeping with the instruction to translate whatever the source actually contains rather than guess at intended content.

Contents

1	Problem Statement	2
1.1	Question 1	2
2	Answers to the Questions	2
2.1	Question 1	2

1 Problem Statement

In this problem, we are asked to use Dijkstra's algorithm to find the shortest path between a start point and an end point, in the presence of obstacles similar to those used in the previous assignment's plane.

1.1 Question 1

Here, we are first asked to read the obstacle information – given in the form of line segments – from a .txt file and plot the obstacles. Then, using Dijkstra's algorithm, we must find the shortest path and plot it.

2 Answers to the Questions

2.1 Question 1

We write the code for the problem in a script as follows (the details of the code will be explained step by step):

```
clc; clear;

dataId = fopen('data1.txt', 'r');
data = fscanf(dataId, '%d', Inf);
fclose(dataId);

obst_no = data(1);
points_data = ones(obst_no, 4);
for i = 1:obst_no
    for j = 1:4
        points_data(i, j) = data(2+4*(i-1)+j-1);
    end
end

[sizeData, ~] = size(data);
P_start = [data(sizeData-3); data(sizeData-2)];
P_end = [data(sizeData-1); data(sizeData)];

X = [points_data(1,1), points_data(1,3)];
Y = [points_data(1,2), points_data(1,4)];
points = [];
polylist = [];
N = 0;
flag = 0; %X does have a number within
for i = 1:obst_no-1
    if flag == 1
        X = [points_data(i+1,1), points_data(i+1,3)];
        Y = [points_data(i+1,2), points_data(i+1,4)];
        flag = 0;
        continue
    end

    if points_data(i,3) == points_data(i+1,1) && points_data(i,4) ==
        points_data(i+1,2)
```

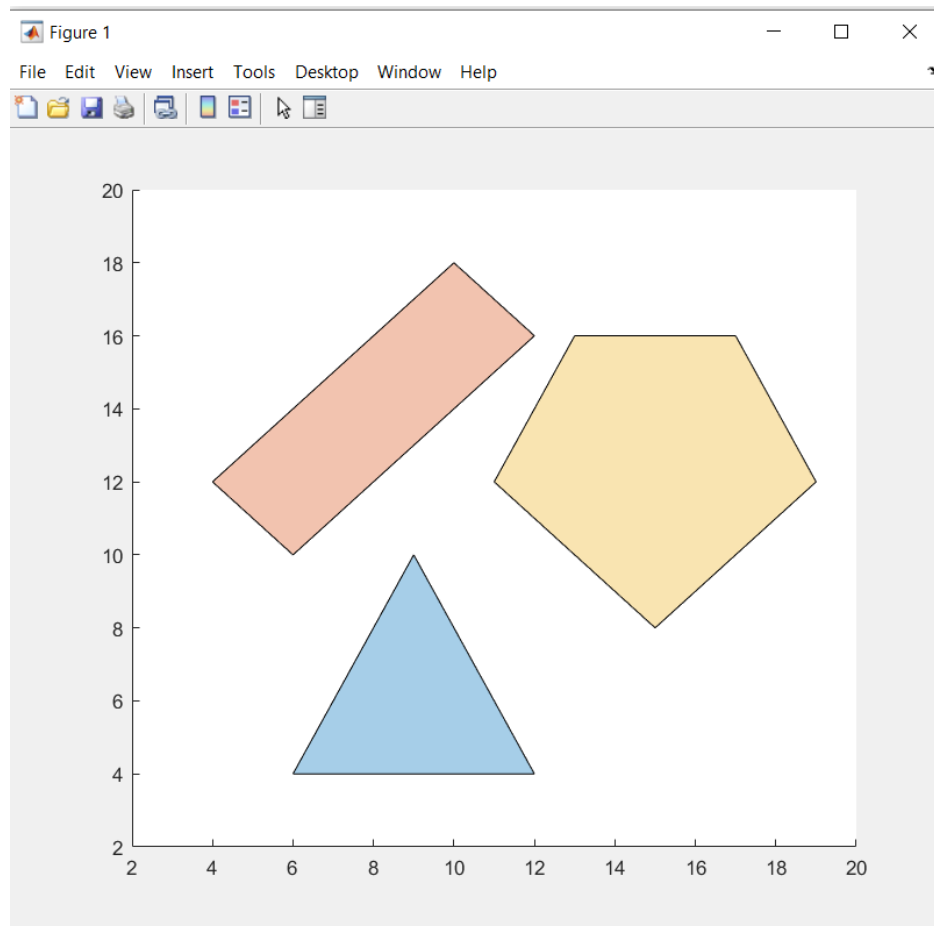
```

        X = [X,points_data(i+1,3)];
        Y = [Y,points_data(i+1,4)];
    end
    if X(end) == X(1) && Y(end) == Y(1)
        X = X(1:end-1);
        Y = Y(1:end-1);
        points = [points;X',Y'];
        N = N+1;
        pgon = polyshape(X,Y);
        polylist = [polylist,pgon];
        plot(pgon);
        hold on
        X = [];
        Y = [];
        flag = 1; %X is empty
    end
end

graph = graph_generator(P_start,P_end, points,polylist,N);
dijkstra(P_start',P_end',points,polylist,N);

```

In the first part of this code, we attempt to organize and structure the data from the `data.txt` file, then extract the obstacles from the information in this file and plot them as polygons.



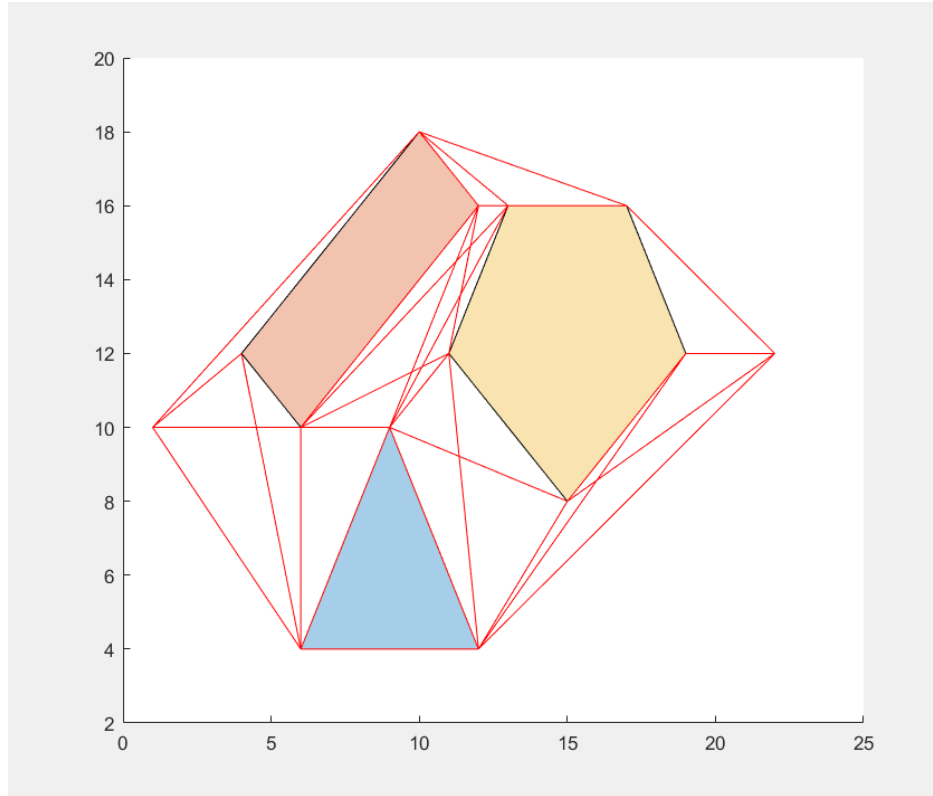
Next, we draw the graph associated with the roadmap method (Road Mapping) using the `graph_generator` function, which plots the graph's edges in red on top of the initial plot for us.

The function and the result of applying it are as follows:

```
function graph = graph_generator(Ps,Pf,points,polylist,N)

allP = [Ps';points;Pf'];
s = size(allP);
number = s(1);
graph = [];
max_graph_lines = factorial(number)/(2*factorial(number-2));

for i = 1:max_graph_lines-1
    for j = 1:max_graph_lines+1
        if i+j <= number
            lineseg = [allP(i,:);allP(i+j,:)];
            IN = [];
            for k = 1:N
                [in,~] = intersect(polylist(k),lineseg);
                IN = [IN,in];
            end
            if isempty(IN)
                lineX = [lineseg(:,1)'];
                lineY = [lineseg(:,2)'];
                plot(lineX,lineY,'r');
                hold on
                graph = [graph; lineX, lineY];
            end
        end
    end
end
end
```



Then, using the `dijkstra` function, the final path is generated and plotted in green on top of the plot.

The function code and its result are as follows:

```
function dijkstra(Ps,Pf,points,polylist,N)

allPoints = flip([Ps;points;Pf]);
[number,~] = size(allPoints);

notMet = remove_row(allPoints,Pf);
X = Pf;
X_previous = [0,0];
Pdata = ones(number,3);
Pdata(1,1:2) = Pf; Pdata(1,3) = 0;
for j = 2:number
    Pdata(j,1:2) = allPoints(j,:);
    Pdata(j,3) = 1000;
end

while X(:) ~= Ps(:)
    neighbors = [];
    dis = [];
    n_neighbors = 0;
    [row,~] = size(notMet);
    for j = 1:row
        lineseg = [X;notMet(j,:)];
        IN = [];
        for k = 1:N
```

```

        [in,~] = intersect(polylist(k),lineseg);
        IN = [IN;in];
    end
    if isempty(IN)
        neighbors = [neighbors;notMet(j,:)];
        n_neighbors = n_neighbors + 1;
        dis = [dis; norm(X-notMet(j,:))];
    end
end
%find current node in Pdata
for j = 1:number
    if X(1) == Pdata(j,1) && X(2) == Pdata(j,2)
        current = j;
        dis_current = Pdata(j,3);
    end
end

for j = 1:number
    for k = 1:n_neighbors
        if Pdata(j,1) == neighbors(k,1) && Pdata(j,2) == neighbors(k,2)
            dis(k) = norm(neighbors(k,:) - X) + dis_current;
            Pdata(j,3) = dis(k);
        end
    end
end

least_distant = 1;
least_distance = dis(1);

for j = 2:n_neighbors
    if dis(j) <= least_distance
        least_distance = dis(j);
        least_distant = j;
    end
end

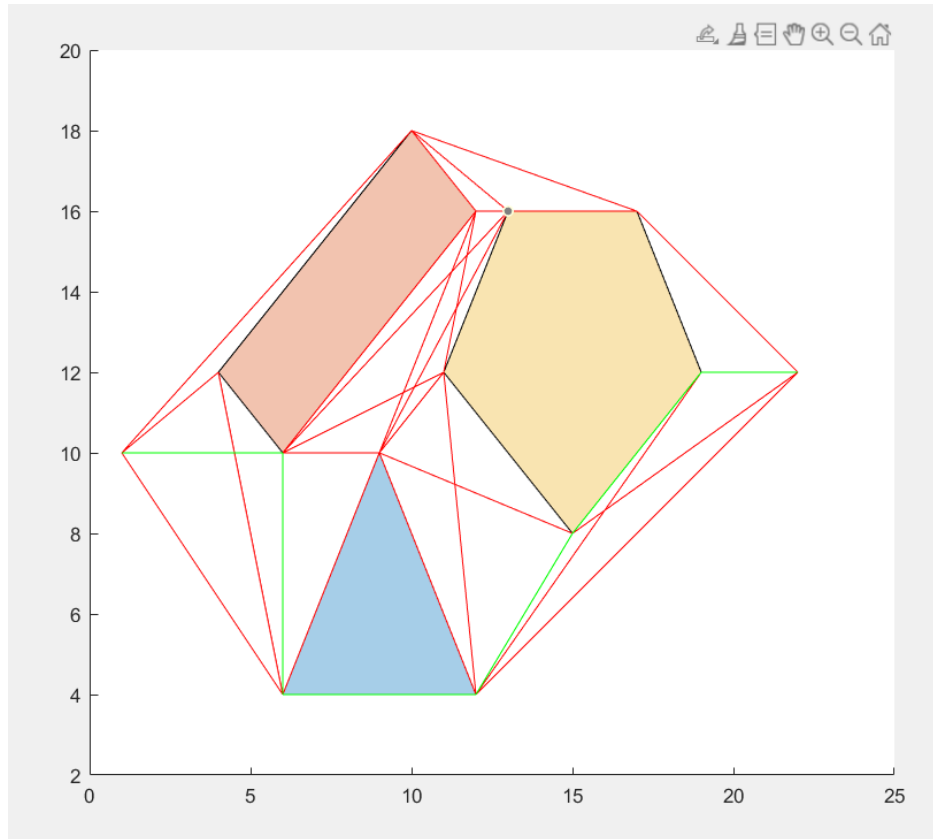
pathX = [X(1), neighbors(least_distant,1)];
pathY = [X(2), neighbors(least_distant,2)];
plot(pathX,pathY,'g');
X_previous = X;
notMet = remove_row(notMet,X_previous);
X = neighbors(least_distant,:);

end

pathX = [X(1), Ps(1)];
pathY = [X(2), Ps(2)];
plot(pathX,pathY,'g');
end

```

In writing this function, another function called `row_remove` is also used, which is used to remove a point from a given dataset.



The code of this function:

```
function mat_modif = remove_row(matrix,row)
[m,~] = size(matrix);
for i = 1:m
    if matrix(i,1) == row(1) && matrix(i,2) == row(2)
        matrix(i,:) = [];
        break;
    end
end
mat_modif = matrix;
end
```

*The codes are included in the appendix.